

Linux_Wk19_S2 Lecture #51

Linux Security Fundamentals

UFW & AppArmor

Table of Contents

Part 1: UFW — Quick Revision

- 1.1 What UFW Does
- 1.2 Essential UFW Commands
- 1.3 What UFW Cannot Do

Part 2: What Is AppArmor?

- 2.1 The Core Idea
- 2.2 Why "Mandatory" Matters
- 2.3 How AppArmor Works — Step by Step
- 2.4 The Two Modes: Enforce and Complain

Part 3: Where AppArmor Fits — The Security Stack

Part 4: AppArmor on Ubuntu — The Basics

- 4.1 AppArmor Is Already Running
- 4.2 Where Profile Files Live
- 4.3 How Profiles Are Named

Part 5: Reading an AppArmor Profile

- 5.1 View the ping Profile
- 5.2 Understand Every Line
- 5.3 File Permission Letters Explained

Part 6: AppArmor Management Commands

Part 7: Practical Labs

- Lab 1: Check AppArmor Status
- Lab 2: Read and Understand a Real Profile
- Lab 3: Create a Custom Profile for Your Own Script
- Lab 4: Switch Between Enforce and Complain Mode

Part 8: Troubleshooting Guide

- 8.1 Reading an AppArmor Denial Log Entry
- 8.2 Common Problems and Solutions
- 8.3 The Standard Debugging Workflow

Part 9: AppArmor vs. Other Security Tools

Part 10: Summary & Quick Reference

- 10.1 Key Concepts to Remember
- 10.2 Commands Quick Reference Card
- 10.3 The Standard Profile Workflow

Part 1: UFW — Quick Revision

1.1 What UFW Does

UFW is a firewall. Think of it as a security guard sitting at the entrance of a building. It checks everyone who tries to enter (incoming network traffic) and controls who is allowed in based on rules you set.

Problem Without UFW	How UFW Fixes It
iptables (the underlying firewall tool) is very complex and hard to use	UFW gives you simple commands like allow and deny
Easy to accidentally block yourself out of the server	UFW defaults to: block all incoming, allow all outgoing
Hard to remember rule syntax	Human-readable commands — no cryptic flags needed

1.2 Essential UFW Commands

Here are all the UFW commands you need to know. You do not need to memorise them — just understand what each one does.

Command	What It Does	When to Use It
<code>sudo ufw status</code>	Shows whether UFW is active and lists current rules	Always check this first
<code>sudo ufw enable</code>	Turns the firewall on	After setting your rules
<code>sudo ufw disable</code>	Turns the firewall off	Troubleshooting only
<code>sudo ufw default deny incoming</code>	Block all incoming traffic by default	Always set this first
<code>sudo ufw default allow outgoing</code>	Allow all outgoing traffic by default	Standard setting
<code>sudo ufw allow 22/tcp</code>	Allow SSH (remote login) connections	Before enabling UFW — or you'll lock yourself out!
<code>sudo ufw allow 80/tcp</code>	Allow normal web traffic (HTTP)	For web servers
<code>sudo ufw allow 443/tcp</code>	Allow secure web traffic (HTTPS)	For web servers
<code>sudo ufw allow from 192.168.1.100</code>	Allow traffic from one specific IP only	Trusted admin machines
<code>sudo ufw delete allow 80/tcp</code>	Remove a rule	When a rule is no longer needed
<code>sudo ufw reset</code>	Delete all rules and disable UFW	Starting over completely

Important: Always allow SSH before enabling UFW

If you are connected to a server via SSH (which you almost always are), run:

```
sudo ufw allow 22/tcp
```

BEFORE running: `sudo ufw enable`

If you enable UFW first without allowing SSH, you will be immediately disconnected and locked out of the server.

1.3 What UFW Cannot Do

This is the most important concept in this section. UFW only watches **network traffic** — it controls who can **connect** to your system. It does **not** watch what programs do after they connect.

UFW Limitation	Real-World Problem This Creates
UFW only controls network traffic	Once a connection is allowed, UFW steps aside completely
UFW does not know which application is listening	Any program can use an allowed port — not just the intended one
UFW cannot restrict file access	A compromised program can read any file the user running it can access
UFW cannot limit what system functions a program calls	Malicious code can exploit the operating system itself

The Gap UFW Leaves Open

Scenario: You run a web server on port 80. UFW allows port 80. Good.

A hacker finds a bug in your web server software and exploits it.

Now the hacker is "inside" — and UFW cannot help you anymore.

The hacker can read your password files, install malware, or connect outward to a remote server.

This is exactly the gap that AppArmor fills.

Part 2: What Is AppArmor?

2.1 The Core Idea

AppArmor is a **Mandatory Access Control (MAC)** system built into the Linux kernel. In simple terms, it assigns each application a **rulebook** — called a **profile** — that defines exactly what that application is allowed to do. Even if the application is compromised, it cannot do anything outside its rulebook.

Security Layer	What It Protects	Real-World Analogy
UFW	Controls network connections — who can connect	Security guard at the building entrance
File permissions	Controls who can read or write specific files	A lock on each office door
AppArmor	Controls what each individual application can do	A personal bodyguard for each employee — follows them everywhere and blocks anything they are not supposed to do

2.2 Why "Mandatory" Matters

In normal Linux security, a user or program can change its own permissions. For example, you can give yourself access to a file you created. This is called Discretionary Access Control — the user has some control.

AppArmor is **mandatory** — meaning the application cannot override its own rules. Even if an attacker takes complete control of the application, the AppArmor profile still limits what damage they can do. Even applications running as root are restricted.

2.3 How AppArmor Works — Step by Step

Here is exactly what happens when you run a program that has an AppArmor profile:

Step	What Happens
1. You run a program	For example: <code>sudo service nginx start</code>
2. Linux checks AppArmor	Linux looks up whether a profile exists for <code>/usr/sbin/nginx</code>
3. Profile is found	AppArmor loads the profile into memory
4. Application runs	nginx starts and begins serving web pages
5. nginx tries to read a file	AppArmor checks: is this file in the profile's allowed list?
6a. File is allowed	nginx reads the file normally. User sees no difference.
6b. File is NOT allowed	AppArmor blocks the read and writes a log entry. nginx gets an error.

2.4 The Two Modes: Enforce and Complain

Every AppArmor profile operates in one of two modes. Understanding these is essential for working with AppArmor.

Mode	What Happens When a Rule Is Violated	When to Use It
Enforce	The action is BLOCKED. The application receives an error. The violation is logged.	Production servers — real security
Complain	The action is ALLOWED but the violation is LOGGED. Nothing is blocked.	Learning and testing — use this when building or debugging a profile

Think of Complain Mode Like a Test Drive

When you create a new profile, you do not know every file and network connection the application needs.

Start in Complain mode — run the application normally, and AppArmor records everything it tries to do.

Review the logs, update the profile to allow legitimate actions, then switch to Enforce mode.

This process is called "profiling" and you will practise it in the labs.

Part 3: Where AppArmor Fits — The Security Stack

Linux security works in layers. Each layer protects against a different type of threat. No single layer is enough on its own — they work together.

UFW — Network Perimeter	Answers: Who is allowed to connect to this machine?
AppArmor — Application Behaviour	Answers: What is this specific program allowed to do?
seccomp — System Call Filter	Answers: Which operating system functions can this program call?
Capabilities — Privilege Limits	Answers: What root-level powers does this program get?
File Permissions — Basic Access	Answers: Who can read or write this file?

We focus on the top two layers: UFW and AppArmor. Together they form a strong defence-in-depth strategy — UFW stops attackers at the door, and AppArmor contains any attacker who gets through.

Part 4: AppArmor on Ubuntu — The Basics

4.1 AppArmor Is Already Running

Good news: AppArmor is installed and enabled by default on Ubuntu. You do not need to install anything to start using it. It is already protecting your system right now.

To check its status, run:

```
sudo aa-status
```

You will see output that looks like this:

```
apparmor module is loaded.  
44 profiles are loaded.  
44 profiles are in enforce mode.  
  /usr/bin/man  
  /usr/sbin/tcpdump  
  ...  
0 profiles are in complain mode.  
0 processes have profiles defined.
```

What this output tells you:

Line in Output	What It Means
apparmor module is loaded	The AppArmor kernel module is active — it is running
44 profiles are loaded	44 applications have AppArmor profiles on this system
44 profiles are in enforce mode	All 44 profiles are actively blocking violations
0 profiles are in complain mode	No profiles are in testing/logging-only mode right now
0 processes have profiles defined	Normal — means none of the profiled apps are running at this exact moment. Not an error.

4.2 Where Profile Files Live

All AppArmor profiles are stored as text files on your system. Knowing where to find them is essential.

Location	What Is Stored There
/etc/apparmor.d/	All active profile files — this is the main folder
/etc/apparmor.d/disable/	Disabled profiles are stored here as symbolic links
/etc/apparmor.d/abstractions/	Reusable rule sets (like shared libraries for profiles)
/var/log/syslog	AppArmor denial messages — check here when something is blocked
/var/log/audit/audit.log	Detailed audit logs (only if the auditd package is installed)

4.3 How Profiles Are Named

Profile files are named after the absolute path of the executable they protect. Slashes (/) and dots (.) in the path are converted to dots in the filename.

Application	Executable Path	Profile File Name
ping	/bin/ping	bin.ping
nginx web server	/usr/sbin/nginx	usr.sbin.nginx
MySQL database	/usr/sbin/mysqld	usr.sbin.mysqld
man (manual pages)	/usr/bin/man	usr.bin.man

How to Find a Profile

To check if an application has a profile, run:

```
sudo aa-status | grep nginx
```

Or look directly in the profile folder:

```
ls /etc/apparmor.d/ | grep nginx
```

If nothing appears, that application does not have an AppArmor profile yet.

Part 5: Reading an AppArmor Profile

A profile is just a text file. Let us read a real one — the profile for the ping command — and understand every part of it.

5.1 View the ping Profile

Run this command to see the profile:

```
sudo cat /etc/apparmor.d/bin.ping
```

You will see something like this:

```
#include <tunables/global>
/bin/ping flags=(complain) {
  #include <abstractions/base>
  #include <abstractions/consoles>
  capability net_raw,
  capability setuid,
  /bin/ping mr,
  /etc/modules.conf r,
  network inet raw,
  network inet6 raw,
}
```

5.2 Understand Every Line

Line	Plain English Explanation
#include <tunables/global>	Load global settings that apply to all profiles. Think of it as importing common definitions.
/bin/ping flags=(complain) {	This profile is for the /bin/ping program. It is currently in complain mode (violations logged, not blocked).
#include <abstractions/base>	Include a reusable set of rules that almost all programs need — like access to basic system libraries. Abstractions save you from writing the same rules in every profile.
#include <abstractions/consoles>	Allow access to terminal/console devices so ping can print output to your screen.
capability net_raw,	Allow ping to use raw network sockets. This is required because ping sends ICMP packets, which need special network access.
capability setuid,	Allow ping to change its user ID temporarily. Some programs need this to perform actions on behalf of different users.
/bin/ping mr,	Allow ping to read (r) its own executable file and memory-map it (m). m is needed to load and execute the binary.
/etc/modules.conf r,	Allow ping to read this configuration file (r = read only). It cannot modify it.
network inet raw,	Allow ping to use IPv4 raw network sockets — needed to send and receive ICMP packets.
network inet6 raw,	Same as above but for IPv6.

5.3 File Permission Letters Explained

When AppArmor grants file access, it uses short letters to say what kind of access is allowed. Here are the most common ones:

Letter	Permission Name	What It Means in Plain English
r	Read	The program can open and read the file, but cannot change it
w	Write	The program can create, modify, or delete the file
m	Memory map	The program can load the file into memory for execution (needed for running programs and shared libraries)
k	Lock	The program can lock the file to prevent other programs from modifying it simultaneously
l	Link	The program can create hard links to the file
ix	Inherit execute	When this program runs another program, the child runs under the same AppArmor profile
px	Profile execute	When this program runs another program, the child runs under that program's own AppArmor profile
ux	Unconfined execute	The child program runs without any AppArmor restrictions. Use with extreme caution.

The Most Common Combinations

- r — Read-only access (configuration files, data files)
- rw — Read and write (log files, data files the program creates)
- mr — Memory map and read (the program's own executable file)
- rw, — Note the comma at the end — every rule in a profile ends with a comma

Part 6: AppArmor Management Commands

Before the labs, here are all the commands you will use to manage AppArmor. The apparmor-utils package provides most of these tools.

Install apparmor-utils First

The profile management tools are NOT installed by default. Install them with:

```
sudo apt install apparmor-utils
```

You only need to do this once.

Command	What It Does	Example
sudo aa-status	Show all loaded profiles and their modes	sudo aa-status grep nginx
sudo aa-enforce <path>	Switch a profile to Enforce mode (blocks violations)	sudo aa-enforce /usr/sbin/nginx
sudo aa-complain <path>	Switch a profile to Complain mode (logs only)	sudo aa-complain /usr/sbin/nginx
sudo aa-disable <path>	Completely disable a profile	sudo aa-disable /usr/sbin/nginx
sudo aa-genprof <path>	Interactively generate a new profile for a program	sudo aa-genprof /home/student/myscript.sh
sudo aa-logprof	Read the logs and suggest profile updates based on what was blocked	sudo aa-logprof
sudo apparmor_parser -r <path>	Reload a profile after you edit it manually	sudo apparmor_parser -r /etc/apparmor.d/bin.ping
sudo apparmor_parser -R <path>	Remove/unload a profile from the kernel	sudo apparmor_parser -R /etc/apparmor.d/bin.ping

Part 7: Practical Labs

Before You Start — Open Two Terminals

Some labs require you to run commands in two terminal windows at the same time.
Right now, open a second terminal window and leave it ready.
We will tell you when to switch between them.

Lab 1: Check AppArmor Status

Goal: Confirm AppArmor is running and find out which programs are protected.

1

Step 1: Check that AppArmor is active

Run:

```
sudo aa-status
```

Look for the line: `apparmor module is loaded`

2

Step 2: Count how many profiles are loaded

In the output from Step 1, find the lines that say:

```
XX profiles are loaded.  
XX profiles are in enforce mode.
```

Make a note of the number. This tells you how many programs AppArmor is currently protecting.

3

Step 3: Check if a specific program has a profile

Search for nginx and mysql profiles:

```
sudo aa-status | grep nginx  
sudo aa-status | grep mysql
```

If nothing appears — that program does not have a profile yet. That is fine.

4

Step 4: Look at what is in the profiles folder

List all profile files:

```
ls /etc/apparmor.d/
```

You will see file names like `usr.sbin.nginx`, `bin.ping`, etc. Each file is one profile.

Lab 1 Complete — What You Learned

AppArmor is already running on your Ubuntu system.
Profiles are text files stored in `/etc/apparmor.d/`
You can see all protected programs with `sudo aa-status`

Lab 2: Read and Understand a Real Profile

Goal: Open an existing profile and understand what it allows.

1

Step 1: View the ping profile

```
sudo cat /etc/apparmor.d/bin.ping
```

This will print the profile to your terminal. Scroll up if needed to see the whole file.

2

Step 2: Find what files ping is allowed to access

Look for lines that mention file paths (they start with / and end with a letter like r, w, or mr).
Example: `/bin/ping mr`, means ping can read and memory-map its own executable.

3

Step 3: Find what capabilities ping has

Look for lines starting with capability.
Example: `capability net_raw`, means ping can send raw network packets.
Refer to the Capabilities table in Part 5 if you need a reminder of what each one means.

4

Step 4: Check the logs for any blocked actions

```
sudo grep apparmor /var/log/syslog | tail -20
```

This shows the last 20 AppArmor log entries. Look for the word DENIED.
If you see DENIED — that means AppArmor blocked something. Note which program and which file path was involved.
If nothing appears — no violations have occurred recently. That is normal.

Lab 2 Complete — What You Learned

Profiles are readable text files — you do not need to be an expert to understand them.

Each rule controls one specific action: read a file, use a network protocol, etc.

You can always check `/var/log/syslog` to see what AppArmor has blocked.

Lab 3: Create a Custom Profile for Your Own Script

Goal: Write a simple script and use aa-genprof to automatically create an AppArmor profile for it. Then enforce the profile and test that it works.

What We Are Building

We will create a simple bash script that reads a file from /tmp/data/ and writes a result to /tmp/output/.

We will then create an AppArmor profile that ONLY allows the script to access those two directories.

Finally, we will prove the profile works by trying to access a file it should NOT be able to read.

1

Step 1: Install apparmor-utils (if not already installed)

```
sudo apt install apparmor-utils
```

If it says 'already installed' — skip to Step 2.

2

Step 2: Create the working directories and input file

```
mkdir -p /tmp/data /tmp/output  
echo 'Hello from AppArmor lab' > /tmp/data/input.txt
```

The first command creates two directories. The second creates a test input file.

3

Step 3: Create the script

```
cat > /home/$USER/myscript.sh << 'EOF'  
#!/bin/bash  
cat /tmp/data/input.txt > /tmp/output/result.txt  
echo 'Script finished. Check /tmp/output/result.txt'  
EOF  
chmod +x /home/$USER/myscript.sh
```

This creates the script file and makes it executable. The script simply copies the input file to the output folder.

4

Step 4: Test the script runs correctly (no AppArmor yet)

```
/home/$USER/myscript.sh
```

You should see: Script finished. Check /tmp/output/result.txt

Verify the output: cat /tmp/output/result.txt

If this does not work, fix the script before continuing.

Step 5: Generate a profile using aa-genprof

In Terminal 1, run:

```
sudo aa-genprof /home/$USER/myscript.sh
```

aa-genprof will start and show you a menu. It is waiting for you to run your script.
The prompt will look like:

```
Profiling: /home/student/myscript.sh

Please start the application to be profiled in
another window and exercise its functionality now.

Once completed, select the (S)can option below in
order to scan the system log for AppArmor events.

[(S)can system log for AppArmor events] / (F)inish
```

5

Do NOT press anything yet. Go to Terminal 2 first.

Step 6: Run the script in Terminal 2

Switch to Terminal 2 and run:

```
/home/$USER/myscript.sh
```

The script will run (possibly with errors — that is OK at this stage).

Now switch back to Terminal 1.

6

Step 7: Scan logs and review suggested rules

In Terminal 1, press S then Enter to scan the logs.

aa-genprof will show you each action the script performed and ask what to do:

```
Profile:      /home/student/myscript.sh
Execute:      /bin/bash

[(I)nherit] / [(C)hild / [(P)rofile / [(N)amed / [(U)nconfined /
[(X)ix On / [(D)eny / Abo(r)t / (F)inish
```

7

For most prompts in this lab, press I (Inherit) then Enter. This means the script runs with the same profile as its parent.

For file access prompts, press A (Allow) to add the rule.

When you have reviewed all suggestions, press F to finish.

Step 8: Save the profile and set it to Enforce mode

aa-genprof will ask if you want to save. Press S to save.

Then set the profile to Enforce mode:

```
sudo aa-enforce /home/$USER/myscript.sh
```

Confirm it is enforced:

```
sudo aa-status | grep myscript
```

You should see the script listed under enforce mode.

8

9

Step 9: Test the enforcement

This command should SUCCEED (it is allowed by the profile):

```
/home/$USER/myscript.sh
```

Now try to do something the profile does NOT allow:

```
# Try to read the password file from within the script context
# Create a test script that tries to access /etc/passwd
echo '#!/bin/bash' > /tmp/badtest.sh
echo 'cat /etc/passwd' >> /tmp/badtest.sh
chmod +x /tmp/badtest.sh
```

Then modify your script temporarily to call this, or simply notice that /etc/passwd is not in the profile.

Check the denial in the logs:

```
sudo grep DENIED /var/log/syslog | tail -10
```

You should see a line containing DENIED and the path that was blocked.

Lab 3 Complete — What You Learned

aa-genprof watches what your application does and builds a profile automatically.

Once enforced, the profile blocks any action not explicitly allowed.

You can always check /var/log/syslog to see exactly what was denied and why.

This is the standard workflow for profiling real applications.

Lab 4: Switch Between Enforce and Complain Mode

Goal: Understand when and how to switch modes. This is a critical skill for troubleshooting.

1

Step 1: Check the current mode of your script's profile

```
sudo aa-status | grep myscript
```

You should see it listed under 'enforce mode'.

2

Step 2: Switch to Complain mode

```
sudo aa-complain /home/$USER/myscript.sh
```

When to use Complain mode:

- You updated the script and it is now doing something new
- Something is broken and you need to find out what AppArmor is blocking
- You are still building the profile and not ready to enforce yet

Confirm the switch:

```
sudo aa-status | grep myscript
```

3

Step 3: Switch back to Enforce mode

```
sudo aa-enforce /home/$USER/myscript.sh
```

Always switch back to Enforce when you are done testing. Complain mode provides no real protection.

4

Step 4: Disable the profile completely

To completely disable a profile (AppArmor will ignore the application):

First, find the profile file name:

```
ls /etc/apparmor.d/ | grep myscript
```

Then disable it (replace the filename with what you found above):

```
sudo aa-disable  
/etc/apparmor.d/home.<yourusername>.myscript.sh
```

To re-enable it:

```
sudo rm  
/etc/apparmor.d/disable/home.<yourusername>.myscript.sh  
sudo apparmor_parser -r  
/etc/apparmor.d/home.<yourusername>.myscript.sh
```

Lab 4 Complete — What You Learned

Enforce mode: violations are blocked. Use this in production.

Complain mode: violations are logged only. Use this when building or debugging profiles.

Disabled: AppArmor ignores the application completely. Avoid this in production.

Part 8: Troubleshooting Guide

When something goes wrong with AppArmor, the answer is almost always in the logs. This section shows you how to find and interpret them.

8.1 Reading an AppArmor Denial Log Entry

A typical denial message in `/var/log/syslog` looks like this:

```
kernel: [1234.567] audit: type=1400 audit(1234567890.123:45) :  
apparmor="DENIED" operation="open" profile="/home/student/myscript.sh"  
name="/etc/shadow" pid=12345 comm="bash" requested_mask="r"  
denied_mask="r"
```

Part of the Log	What It Tells You
apparmor="DENIED"	AppArmor blocked this action
operation="open"	The program tried to open a file
profile="/home/student/myscript.sh"	This profile was active when the block occurred
name="/etc/shadow"	This is the file the program tried to access
requested_mask="r"	The program wanted read access
denied_mask="r"	Read access was denied by the profile

8.2 Common Problems and Solutions

Problem	Symptom	Solution
Program fails with a mysterious error	No obvious error message — program just does not work	Check: <code>sudo grep DENIED /var/log/syslog tail -20</code> Look for your program's name
Profile is too restrictive	Even normal operations are blocked	Switch to Complain mode, run the program, review logs, update the profile, switch back to Enforce
Profile file not loading	<code>sudo aa-status</code> does not show your profile	Run: <code>sudo apparmor_parser -r /path/to/profile</code> Check for syntax errors in the profile file
Locked out of important tool	Something critical stopped working after adding AppArmor	Find the profile: <code>sudo aa-status grep programname</code> Temporarily: <code>sudo aa-complain /path/to/program</code>
aa-genprof not found	Command not found error	Run: <code>sudo apt install apparmor-utils</code>

8.3 The Standard Debugging Workflow

When AppArmor is causing a problem, follow these steps in order:

1

Step 1: Check the logs first

```
sudo grep DENIED /var/log/syslog | tail -30
```

Why: Find exactly what AppArmor is blocking

2

Step 2: Identify the profile

```
sudo aa-status | grep <programname>
```

Why: Confirm which profile is causing the block

3

Step 3: Switch to Complain mode

```
sudo aa-complain <profile path>
```

Why: Stop the blocking temporarily so you can test

4

Step 4: Run the application

```
Run it normally, trigger the action that was failing
```

Why: Reproduce the problem while AppArmor is logging

5

Step 5: Update the profile

```
sudo aa-logprof
```

Why: Let AppArmor suggest which rules to add

6

Step 6: Switch back to Enforce

```
sudo aa-enforce <profile path>
```

Why: Re-enable protection with the updated profile

7

Step 7: Test again

```
Repeat the action that was failing before
```

Why: Confirm it works and AppArmor is happy

Part 9: AppArmor vs. Other Security Tools

AppArmor is one of several security tools available on Linux. Here is how they compare so you know which to use in different situations.

Tool	What It Protects	Best Used For
UFW	Network — controls incoming and outgoing connections	Allowing or blocking specific ports and IP addresses
AppArmor	Application behaviour — controls what a program can access and do	Containing the damage if a program is exploited
SELinux	Application behaviour — similar to AppArmor but label-based (used on Red Hat/CentOS)	Enterprise Red Hat environments
seccomp	System calls — controls which kernel functions a program can use	Container security, advanced hardening
File permissions	File access — controls who can read/write individual files	Basic file security (always the first line of defence)
chroot	Filesystem view — isolates a program to a directory subtree	Legacy tool — limited protection on its own, not recommended as a primary security boundary

Key Takeaway: Use Layers, Not Just One Tool

UFW + AppArmor + proper file permissions together provide much stronger security than any one alone.

Think of it as: UFW keeps attackers out, AppArmor limits damage if one gets in, file permissions protect your data.

On Ubuntu, start with UFW and AppArmor — they are built-in and well supported.

Part 10: Summary & Quick Reference

10.1 Key Concepts to Remember

Term	Definition
AppArmor	A Linux security system that restricts what applications can do, even if they are compromised
Profile	A text file that defines the rules (allowed files, network, capabilities) for one application
Enforce mode	Violations are BLOCKED. Use this in production environments.
Complain mode	Violations are LOGGED but allowed. Use this when building or debugging profiles.
Abstraction	A reusable set of rules included in profiles with #include. Saves repeating common rules.
Capability	A specific Linux privilege (like using raw network sockets or changing user ID)
DAC	Discretionary Access Control — standard Linux permissions. Users can control their own files.
MAC	Mandatory Access Control — AppArmor's approach. Policies cannot be overridden by users or programs.

10.2 Commands Quick Reference Card

Task	Command
Check AppArmor status	<code>sudo aa-status</code>
Check if a program is profiled	<code>sudo aa-status grep <programname></code>
View a profile file	<code>sudo cat /etc/apparmor.d/<profilename></code>
Set a profile to Enforce mode	<code>sudo aa-enforce /path/to/program</code>
Set a profile to Complain mode	<code>sudo aa-complain /path/to/program</code>
Generate a new profile interactively	<code>sudo aa-genprof /path/to/program</code>
Update profile from log	<code>sudo aa-logprof</code>
Reload a profile after editing	<code>sudo apparmor_parser -r /etc/apparmor.d/<profilename></code>
View recent AppArmor denials	<code>sudo grep DENIED /var/log/syslog tail -20</code>
View all AppArmor log entries	<code>sudo grep apparmor /var/log/syslog tail -20</code>

10.3 The Standard Profile Workflow

Stage	Action	Mode
1. Generate	Run aa-genprof and exercise the application	Complain
2. Review	Run aa-logprof to review and accept suggested rules	Complain
3. Test	Run the application fully — confirm it works as expected	Complain
4. Enforce	Run aa-enforce to activate full protection	Enforce
5. Monitor	Check /var/log/syslog periodically for unexpected denials	Enforce
6. Update (when needed)	Switch to Complain, update with aa-logprof, switch back to Enforce	Enforce